

Entrega Final

Created by  Andrea

Andrea Carolina Rodríguez Salinas

Código: 202214026

Índice General

1. Idea General y Objetivo del Proyecto
 2. Entrega 1 – Modelado y Preparación Inicial
 3. Entrega 2 – Corrección de Errores, Animación y Código
 4. Entrega 3 – Cambio de Enfoque e Integración Ray Tracing
 5. Propuesta Final y Extensión del Proyecto
 6. Recursos y Herramientas Utilizadas
 7. Repositorio
 8. Referencias
-

1. Idea General y Objetivo del Proyecto

La idea inicial consistió en desarrollar un oso de goma interactivo en realidad aumentada. El objetivo fue integrar este personaje en un entorno de realidad aumentada usando Diligent Engine y lograr que tenga interacciones, texturas realistas y comportamientos animados como caminar, engordar al recibir corazones o permitir que el usuario lo pinte (idea inspirada en *Open Brush*).

El proyecto se concibió como una extensión significativa del ejemplo *Hello AR* de Diligent Engine, reemplazando los cubos con un oso animado y con estética visual basada en Ray Tracing (Tutorial 21).

2. Entrega 1 – Modelado y Preparación Inicial

- Se modeló el oso de goma en Blender con textura gelatinosa.
 - Se integró al ejemplo GLTF de Diligent encontrando problemas como pérdida de texturas y geometría incompleta.
 - Se propuso una solución modificando el *loader* para cargar texturas directamente sin `loadModel()`.
 - Se seleccionó una textura cristalina inspirada en el Tutorial 21 y se revisaron shaders relacionados a absorción de luz y reflexión.
-

3. Entrega 2 – Corrección de Errores, Animación y Código

- **Errores corregidos:** Se corrigieron errores de visualización del oso dentro del GLTFViewer (orejas, brazos, piernas completas y textura cargada).
 - **Animación:** Usando rigging, se animó una caminata simple del oso dentro de Diligent (movimiento de brazos, caderas, piernas y orejas).
 - **Código:**
 - Se modificó el archivo `DefaultGLTFModels[]` para incluir el oso.
 - Se limpió la función `Update()` eliminando cargas innecesarias.
 - Se logró una animación reproducible dentro de Diligent.
 - **Investigación AR:**
 - Se contactó con Colivri para usar gafas de realidad aumentada.
 - Se comenzó a explorar ARCore y compatibilidad con ejemplos Android de Diligent.
-

4. Entrega 3 – Cambio de Enfoque e Integración Ray Tracing

Debido a limitaciones de tiempo y conocimientos en desarrollo móvil, el enfoque cambió a una escena usando el *Tutorial 21* de Ray Tracing.

Cambios Hechos en la idea principal:

- La escena debería contener:
 - Una mesa modelada en Blender (estática).
 - Cajas generadas en Diligent con textura simulando cartón (dulces Oka Loka).
 - Osos caminantes que se dirigen a la caja más cercana y desaparecen al contacto.

Avances:

- **Mesa:** Modelada y cargada correctamente en Diligent.
- **Cajas:**
 - Basadas en tarea 11.
 - Modificado para usar cubos en vez de esferas.
 - Añadido control por slider para aumentar/disminuir número de cajas.
- **Textura:**
 - Textura de cartón creada con apariencia opaca.
 - Shader editado para aplicar correctamente la textura.
- **Integración con GLTFViewer:**

- Se intentó integrar mediante funciones `cpp/hpp`, pero aparecieron conflictos entre ejemplos y dependencias cruzadas.

5. Propuesta Final y Extensión del Proyecto

Se propone como trabajo final:

- Mostrar múltiples osos caminando en una mesa hacia cajas de dulces.
- Usar Ray Tracing para simulación visual avanzada.

6. Entrega Final

Objetivos y motivacion

Objetivo

Desarrollar una aplicación en realidad aumentada que permita la visualización e interacción de un oso tridimensional animado, modelado desde cero, el cual se desplace de manera autónoma hacia objetivos definidos dentro de un entorno simulado, superpuesto al espacio físico del usuario.

Motivaciones:

La motivación central de este proyecto radica en la intención de explorar las posibilidades de la realidad aumentada como medio para integrar modelos tridimensionales animados en entornos físicos reales. Particularmente, surge del interés por desarrollar un personaje virtual, en este caso, un oso capaz de interactuar con su entorno mediante desplazamientos autónomos. Esta iniciativa combina dos áreas de profundo interés: la animación 3D mediante herramientas como Blender y la implementación de sistemas interactivos en realidad aumentada.

Para la entrega final se trato de integrar las dos partes del codigo (tutorial 21 y GLTF_viewer)

para esto se intentaron diferentes formas las cuales se van a enumerar a continuacion

1. inicialmente se trato de integrar el tutorial 21 con el gltfviewer generando las mismas funciones del gltf dentro del tutorial, esto no funciono debido a que saltaban diferentes errores de enrutamiento, igualmente al hacerlo al contrario es decir integrar el gltf viewer con las funciones hechas en el gltfviewer esto genero un problema severo al momento de tratar de integrarlos ocurrían los mismos errores

```

36 #include "CommonlyUsedStates.h"
37 #include "ShaderMacroHelper.hpp"
38 #include "FileSystem.hpp"
39 #include "imgui.h"
40 #include "../imguiZMO.quat/imguiZMO.h"
41 #include "ImGuiUtils.hpp"
42 #include "CallbackWrapper.hpp"
43 #include "CommandLineParser.hpp"
44 #include "GraphicsAccessories.hpp"
45 #include "EnvMapRenderer.hpp"
46 #if
47 // no se puede abrir el archivo origen (código de error "EnvMapRenderer.hpp").
48 // Buscar en línea
49 //
50 #include "Utilities/Interface/DiligentFXShaderSourceStreamFactory.hpp"
51 #include "ShaderSourceFactoryUtils.hpp"
52
53 namespace Diligent
54 {
55
56 namespace HLSL

```

Figura 1

Para solucionar estos errores se importo el cpp en el entorno de diligent donde se encontraba el gltf viewer al hacer esto se logro realizar el enrutamiento de manera exitosa y se pudo conectar ambas partes del trabajo. para esto donde se encontraba el `GLTFViewer.cpp` se tuvo que importar `SampleBase.cpp` y `Tutorial21.cpp` con estos dos se logro que ambos entornos corrieran al mismo tiempo al usar funciones dentro del tutorial 21 `LoadModel()` `UpdateUI()` `Update()` esto genero errores tales como los observados en la figura 2, para solucionar esto se realizaron arreglos dentro de las funciones para que estas funcionaran simultáneamente

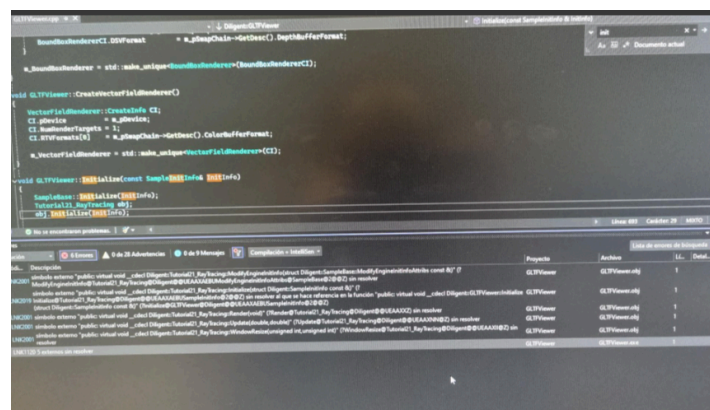


Figura 2

Lastimosamente al integrar ambos entornos se sobre ponían los dos modelos haciendo que solo se viera el definido inicialmente y se observara solamente la UI de el otro

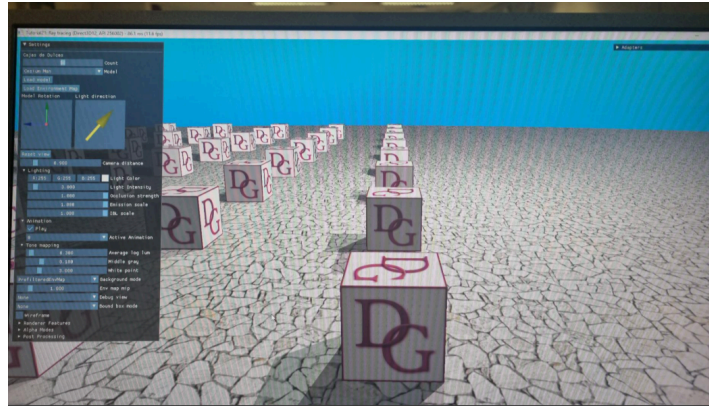


Figura 3

Codigo resumido del codigo es el siguiente:

```
GLTFViewer obj;
void Tutorial21_RayTracing::Initalize(){
obj.Initialize(InitInfo);
obj.LoadModel("models/CesiumMan/gltf/CesiumMan.gltf");
obj.UpdateScene();
}
```

2. Para solucionar los inconvenientes anteriormente mencionados se penso en otro enfoque tal como generar funciones adicionales dentro del `Tutorial21.cpp` en este se querian integrar funciones que simularan

- a. la carga de la geometria `CreateGLTFBLAS()`
- b. editar las funciones como `Initialize()` `CreateSBT()` `UpdateTLAS()` `Update()` `Render()`

```
void Tutorial21_RayTracing::CreateGLTFBLAS()
{
    if (!m_pGLTFModel)
        return;

    // Create BLAS for each mesh in the GLTF model
    std::vector<BLASTriangleDesc> triangles;
    std::vector<RefCntAutoPtr<IBuffer>> vertexBuffers;
    std::vector<RefCntAutoPtr<IBuffer>> indexBuffers;
    std::vector<BLASBuildTriangleData> triangleData;

    for (const auto& mesh : m_pGLTFModel->Meshes)
    {
        for (const auto& primitive : mesh.Primitives)
        {
```

```

BLASTriangleDesc triangleDesc;
triangleDesc.GeometryName      = ("GLTFMesh_" + std::to_string(mesh.Id) + "_" + std::
triangleDesc.MaxVertexCount    = primitive.NumVertices;
triangleDesc.VertexValueType    = VT_FLOAT32;
triangleDesc.VertexComponentCount = 3;
triangleDesc.MaxPrimitiveCount  = primitive.NumIndices / 3;
triangleDesc.IndexType          = VT_UINT32;
triangles.push_back(triangleDesc);

// Create vertex and index buffers
RefCntAutoPtr<IBuffer> vertexBuffer;
RefCntAutoPtr<IBuffer> indexBuffer;
GeometryPrimitiveBuffersCreateInfo buffersCI;
buffersCI.VertexBufferBindFlags = BIND_RAY_TRACING;
buffersCI.IndexBufferBindFlags  = BIND_RAY_TRACING;

BufferDesc vertBuffDesc;
vertBuffDesc.Name      = ("GLTF Vertex Buffer " + std::to_string(mesh.Id)).c_str();
vertBuffDesc.Usage      = USAGE_DEFAULT;
vertBuffDesc.BindFlags  = BIND_RAY_TRACING;
vertBuffDesc.Size       = primitive.NumVertices * sizeof(float3);
m_pDevice->CreateBuffer(vertBuffDesc, nullptr, &vertexBuffer);

BufferDesc idxBuffDesc;
idxBuffDesc.Name        = ("GLTF Index Buffer " + std::to_string(mesh.Id)).c_str();
idxBuffDesc.Usage        = USAGE_DEFAULT;
idxBuffDesc.BindFlags    = BIND_RAY_TRACING;
idxBuffDesc.Size         = primitive.NumIndices * sizeof(UInt32);
m_pDevice->CreateBuffer(idxBuffDesc, nullptr, &indexBuffer);

vertexBuffers.push_back(vertexBuffer);
indexBuffers.push_back(indexBuffer);

BLASBuildTriangleData triData;
triData.GeometryName      = triangleDesc.GeometryName;
triData.pVertexBuffer      = vertexBuffer;
triData.VertexStride       = sizeof(float3);
triData.VertexCount        = triangleDesc.MaxVertexCount;
triData.VertexValueType     = triangleDesc.VertexValueType;
triData.VertexComponentCount = triangleDesc.VertexComponentCount;
triData.pIndexBuffer        = indexBuffer;
triData.PrimitiveCount      = triangleDesc.MaxPrimitiveCount;
triData.IndexType           = triangleDesc.IndexType;

```

```

        triData.Flags          = RAYTRACING_GEOMETRY_FLAG_OPAQUE;
        triangleData.push_back(triData);
    }
}

// Create BLAS
BottomLevelASDesc ASDesc;
ASDesc.Name          = "GLTF BLAS";
ASDesc.Flags         = RAYTRACING_BUILD_AS_PREFER_FAST_TRACE;
ASDesc.pTriangles    = triangles.data();
ASDesc.TriangleCount = static_cast<Uint32>(triangles.size());

m_pDevice->CreateBLAS(ASDesc, &m_pGLTFBLAS);
VERIFY_EXPR(m_pGLTFBLAS != nullptr);

// Create scratch buffer
RefCntAutoPtr<IBuffer> pScratchBuffer;
{
    BufferDesc BuffDesc;
    BuffDesc.Name      = "GLTF BLAS Scratch Buffer";
    BuffDesc.Usage     = USAGE_DEFAULT;
    BuffDesc.BindFlags = BIND_RAY_TRACING;
    BuffDesc.Size      = m_pGLTFBLAS->GetScratchBufferSizes().Build;

    m_pDevice->CreateBuffer(BuffDesc, nullptr, &pScratchBuffer);
    VERIFY_EXPR(pScratchBuffer != nullptr);
}

// Build BLAS
BuildBLASAttribs Attribs;
Attribs.pBLAS          = m_pGLTFBLAS;
Attribs.pTriangleData  = triangleData.data();
Attribs.TriangleDataCount = static_cast<Uint32>(triangleData.size());
Attribs.pScratchBuffer = pScratchBuffer;
Attribs.BLASTransitionMode = RESOURCE_STATE_TRANSITION_MODE_TRANSITION;
Attribs.GeometryTransitionMode = RESOURCE_STATE_TRANSITION_MODE_TRANSITION;
Attribs.ScratchBufferTransitionMode = RESOURCE_STATE_TRANSITION_MODE_TRANSITION;

m_pImmediateContext->BuildBLAS(Attribs);
}

```

La función

`CreateGLTFBLAS()` tiene como propósito construir la **estructura BLAS** para un modelo GLTF cargado, necesaria para que los objetos del modelo puedan ser utilizados en **Ray Tracing** con Diligent.

- a. primero se verifica que el modelo este cargado
- b. se declaran variables auxiliares que ayudan al proceso del ray tracing
- c. procesa dada malla del modelo y se le asignan nombres a los vertices, tipo de dato, cantidad de triangulos entre otros
- d. creacion de buffers de vertice e indices
- e. condifuracion el BLASBuildTriangleData
- f. se crean los objetos BLAS
- g. Se crean buffers , es una memoria para cosntruir el BLAS

la idea de este codigo era poder crear el modelo rastreado

este fallo debido a errores que aparecían que eran de tipos de datos cuando se usaban dentro de otros códigos y no generaban error por lo que no se pudo resolver el error y se trato de hacer lo mismo con diferentes funciones para que funcionara pero no se logro este fue el intento más cercano al exito.

```
CreateGraphicsPSO();
CreateRayTracingPSO();
LoadTextures();
CreateCubeBLAS();
CreateProceduralBLAS();
CreateGLTFBLAS(); // se añadio esta funcion en initialize
UpdateTLAS();
CreateSBT();
```

3. Posteriormente, al estar llegando al limite del tiempo se decidio por tomar otro enfoque que llevara a un resultado similar, este enfoque seria realizar una caja la cual hiciera las acciones que queríamos que hiciera el oso al momento de aparecer en el tutorial estas actividades se resumen en tratar de hacer que el oso camine hacia la caja mas cercana para esto se va a mostrar el codigo "Mas exitoso" este genera el video visto en el readme del repositorio para esto se crearon

- a. variables nuevas

```
private:
    float3 m_Sphere4Pos = float3(8.0f, 0.0f, 0.0f); // Posición inicial de la instancia 4
    float m_LastFrameTime = 0.0f; // Tiempo del último frame para el cálculo de velocidad
```

1. se actualizo el update para mostrar el ultimo frame donde estuvo la bolita


```

void Tutorial21_RayTracing::Update(double CurrTime, double ElapsedTime)
{
    SampleBase::Update(CurrTime, ElapsedTime);
    UpdateUI();

    m_LastFrameTime = static_cast<float>(ElapsedTime); // Almacenar tiempo del frame
    // ... resto del código
}

```

c. Se modificaba la posición de las esferas en el tlas para poder hacer que estas se movieran

```

for (int i = 0; i < MaxSmallSpheres; ++i)
{
    int idx = 5 + i;
    auto& inst = Instances[idx];

    smallSphereNames[i] = "SmallSphere" + std::to_string(i);
    inst.InstanceName = smallSphereNames[i].c_str();

    inst.pBLAS = m_pCubeBLAS;
    inst.Mask = (i < m_NumScatterSpheres ? OPAQUE_GEOM_MASK : 0u);
    inst.CustomId = 3;

    if (i < m_NumScatterSpheres && !m_ScatterSpherePositions.empty())
    {
        const auto& pos = m_ScatterSpherePositions[i];
        inst.Transform.SetTranslation(pos.x, pos.y, pos.z);
    }
    else
    {
        inst.Transform.SetTranslation(0.0f, -1000.0f, 0.0f); // Mover fuera de vista si no está activa
    }
}

```

En teoría la idea de el código era que al cargar el tutorial 21 una bolita persiguiera a la caja mas cercana y que estas También se pudieran mover al rededor del mapa pero este genero la vista hecha en video puesto en el Readme

7. Repositorio

https://github.com/acrodriguezs1/Proyecto_Final_CVI

8. Analisis de codigo Extendido

a. Archivos involucrados

a. codigo base

GLTFViewer.cpp

GLTFViewer.hpp

b. Codigo extendido

Tutorial21_RayTracing.cpp

Tutorial21_RayTracing.hpp

b. Analisis comparativo

- numero de lineas

GLTFViewer.cpp 1360 lineas

GLTFViewer.hpp 210 lineas

Tutorial21_RayTracing.cpp 1440 lineas

Tutorial21_RayTracing.hpp 180 lineas

- Clases principales

- Tutorial21_Raytracing
- GLTFViewer

- Extencion de la arquitectura

- CreateGLTFBLAS
- modelo del GLTF
- Renderizado
- Animaciones
- envmaprender
- UpdateTLAS

- Conexion entre ambos modulos

- GLTFViewer **importa** Tutorial21_RayTracing.hpp , se reutilizan estructuras en comun

- Sin embargo, cada uno actúa como una **independiente** (con su propio Initialize() , Render() , etc.).

En las otras dos formas tienen una estructura similar

ademas se realizo un gltfPrimary hit para hacer la simulacuon de los rayos sobre una estructura como shader adicional

9. Referencias

- <https://github.com/DiligentGraphics/DiligentSamples/tree/master/Samples/GLTFViewer>
- <https://github.com/DiligentGraphics/DiligentSamples/tree/master/Android/HelloAR>
- https://github.com/DiligentGraphics/DiligentSamples/tree/master/Tutorials/Tutorial21_RayTracing
- <https://github.com/google-ar/arcore-android-sdk>
- <https://www.blender.org/>

La motivación central de este proyecto radica en la intención de explorar las posibilidades de la realidad aumentada como medio para integrar modelos tridimensionales animados en entornos físicos reales. Particularmente, surge del interés por desarrollar un personaje virtual, en este caso, un oso capaz de interactuar con su entorno mediante desplazamientos autónomos. Esta iniciativa combina dos áreas de profundo interés: la animación 3D mediante herramientas como Blender y la implementación de sistemas interactivos en realidad aumentada.